

Resume AI Assistant

API Documentation

1. Upload Resume

API Name	Upload Resume
URL / Endpoint	/upload
Integration Type	REST
Method	POST
Description	Uploads a PDF resume, saves it on the server, and builds (or rebuilds) the RAG pipeline — embeddings + retriever — used to answer questions about it via the /ask endpoint.
Module / Service	Resume Q&A Service (RAGPipeline)
Authorization	None (local demo API, no authentication required)
Headers	Content-Type: multipart/form-data (set automatically by the HTTP client)
Content-Type	multipart/form-data
Request Parameters	None. The file is sent as the request body.
Request Body	form field "file": PDF file (application/pdf)
Sample Request	<pre>curl -X POST http://localhost:8000/upload \ -F "file=@resume.pdf;type=application/pdf"</pre>
Success Response	<pre>{ "message": "Resume 'resume.pdf' uploaded and indexed successfully." }</pre>
Error Responses	<pre>400 Bad Request { "detail": "Only PDF files are supported." } 400 Bad Request { "detail": "<PDF read error, e.g. empty or unreadable file>" }</pre>
Output Fields	message (string) — confirmation message including the uploaded file's name.
Notes	Only one resume is active for Q&A at a time; uploading a new file replaces the one used by /ask. The first request after server startup can take longer because the embedding model is loaded into memory.

2. Ask Question

API Name	Ask Question
URL / Endpoint	/ask
Integration Type	REST
Method	POST
Description	Answers a natural-language question about the currently uploaded resume. Checks relevance with cosine similarity, then generates the answer with Qwen3 (via Ollama) if the question is in scope.
Module / Service	Resume Q&A Service (RAGPipeline / Generator)
Authorization	None (local demo API, no authentication required)
Headers	Content-Type: application/json
Content-Type	application/json
Request Parameters	None.
Request Body	<pre>{ "question": "string" }</pre>
Sample Request	<pre>curl -X POST http://localhost:8000/ask \ -H "Content-Type: application/json" \ -d '{"question": "What programming languages does the candidate know?"}'</pre>
Success Response	<pre>{ "answer": "The candidate knows Python and R." }</pre>
Error Responses	<pre>400 Bad Request { "detail": "Please upload a resume first." } 400 Bad Request { "detail": "Question cannot be empty." } 500 Internal Server Error { "detail": "<generation error message>" }</pre>
Output Fields	answer (string) — the model's answer, or a fixed message if the question is out of scope of the resume.
Notes	If the question's similarity to the resume text falls below the relevance threshold (0.2), the API returns "I can only answer questions about the resume." without calling the language model.

3. Upload Resumes for Matching

API Name	Upload Resumes for Matching
URL / Endpoint	/match/upload
Integration Type	REST
Method	POST
Description	Uploads one or more PDF resumes and adds them to the shared resume pool used by the Job Matcher.
Module / Service	Job Matcher Service (JobMatcher)
Authorization	None (local demo API, no authentication required)
Headers	Content-Type: multipart/form-data (set automatically by the HTTP client)
Content-Type	multipart/form-data
Request Parameters	None. Files are sent as the request body.
Request Body	form field "files": one or more PDF files (repeated field, application/pdf)
Sample Request	<pre>curl -X POST http://localhost:8000/match/upload \ -F "files=@resume1.pdf;type=application/pdf" \ -F "files=@resume2.pdf;type=application/pdf"</pre>
Success Response	<pre>{ "message": "Uploaded 2 resume(s) for matching.", "filenames": ["resume1.pdf", "resume2.pdf"] }</pre>
Error Responses	400 Bad Request <pre>{ "detail": "No valid PDF files were uploaded." }</pre>
Output Fields	message (string) — upload confirmation. filenames (array of strings) — names of the files that were saved successfully.
Notes	Non-PDF files in the request are silently skipped rather than failing the whole request. Saved files persist in the resume pool and are included in every subsequent /match call.

4. Match Job Description

API Name	Match Job Description
URL / Endpoint	/match
Integration Type	REST
Method	POST
Description	Compares a job description against every resume in the resume pool and returns the single best-matching resume, based on embedding cosine similarity.
Module / Service	Job Matcher Service (JobMatcher)
Authorization	None (local demo API, no authentication required)
Headers	Content-Type: application/json
Content-Type	application/json
Request Parameters	None.
Request Body	<pre>{ "job_description": "string" }</pre>
Sample Request	<pre>curl -X POST http://localhost:8000/match \ -H "Content-Type: application/json" \ -d '{"job_description": "Looking for a candidate with strong Python and data science skills"}'</pre>
Success Response	<pre>{ "filename": "resume1.pdf", "resume_text": "<full extracted resume text>" }</pre>
Error Responses	<pre>400 Bad Request { "detail": "Job description cannot be empty." } 400 Bad Request { "detail": "No resumes found. Please upload at least one resume." }</pre>
Output Fields	filename (string) — name of the best-matching resume file. resume_text (string) — full extracted text of that resume.
Notes	All resumes in the pool are re-embedded on every request (no caching), so response time grows with the number and length of resumes.